

1. The result displayed from `console.log(name)` is?



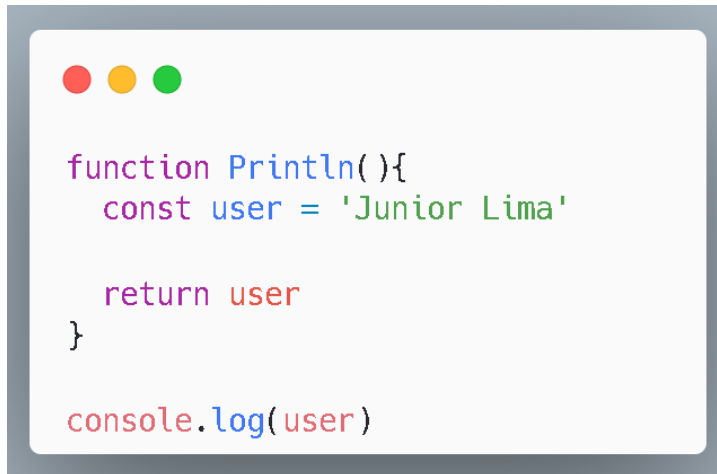
```
let name = 'Dedi'  
name += 'Kusnandar'  
name = 'Ciro Alves'  
console.log(name)
```

- a. Dedi
- b. Kusnandar
- c. Ciro Alves
- d. Dedi Kusnandar Ciro Alves

2. LIFO (Last in First Out) is a concept used in one type of data structure. The data structure that uses LIFO is:

- a. Stack
- b. Queue
- c. Hash Map/Hash Table
- d. LinkedList

3. What will be displayed by `console.log` if the code below is executed?



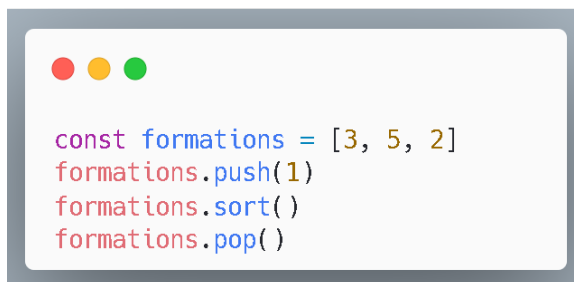
```
function Println(){
  const user = 'Junior Lima'

  return user
}

console.log(user)
```

- a. Undefined
- b. Null
- c. Junior Lima
- d. `ReferenceError` user is not defined

4. What the final value of variable `formations`?



```
const formations = [3, 5, 2]
formations.push(1)
formations.sort()
formations.pop()
```

- a. [1, 2, 3]
- b. [2, 3, 5]
- c. [5, 2, 1]
- d. [1, 2, 5]

5. What is current value of variable result?



```
let result = 'Hello'  
  
if('0' !== 0) result += ', World!'
```

- a. 'Hello'
- b. ', World!'
- c. 'Hello, World!'
- d. TypeError: Assignment to constant variable

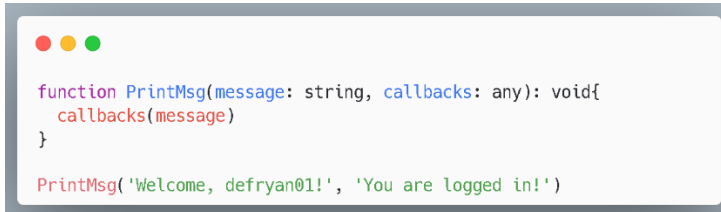
6. The following are correct statements regarding Array in JavaScript, except?

- a. Has `.length`` method
- b. It can accommodate more than one value
- c. It can hold all data types and values in javascript, including themselves
- d. It can hold all data types and values in javascript except for object and function

7. The difference between `while` and `do while` in javascript is:

- a. In a while loop, the loop will only run if the given condition is false, whereas in a do while loop, it will continue to run continuously even if the given condition is true
- b. In a while loop, the condition is checked after each iteration before executing the statements inside the while loop, whereas in a do while loop, the condition is checked first before each iteration
- c. A while loop will only run if the given condition is true, whereas a do while loop will continue to run continuously even if the given condition is false
- d. In a while loop, the condition is checked before each iteration before executing the statements inside the while loop, whereas in a do while loop, the condition is checked after the iteration has run

8. What will happen if the following program is executed?



```
function PrintMsg(message: string, callbacks: any): void{  
  callbacks(message)  
}  
  
PrintMsg('Welcome, defryan01!', 'You are logged in!')
```

- a. `Welcome, defryan01`
- b. `You are logged in!`
- c. `TypeError: callback is not a function`
- d. `ReferenceError: callback is not defined`

9. The result of the code bellow when executed is:

```
const name: string = 'Indivara'
const age: number = 17

if(!name || !umur) console.log('Data not valid')
else if(name && umur > 17) console.log('Verification success!')
else if(name && umur < 17) console.log('Verification failed!')
else console.log('Verification on process!')
```

- a. Data not valid
- b. Verification success!
- c. Verification failed!
- d. Verification on process!

10. What result would be displayed if we execute the command `console.log(nmbr)` from the variable data in the last line, and what is its data type, `typeof nmbr` ?

```
let nmbr: number | string = '12z'
nmbr = Number(nmbr)
nmbr = String(nmbr)
```

- a. NaN and String
- b. '12' and String
- c. NaN and Number
- d. 12 and Number

11. What the result of console bellow?



```
const number = 'WD123'  
console.log(parseInt(number))
```

- a. NaN
- b. 123
- c. '123'
- d. NaNNaN123

12. What the result of console bellow?



```
const number = '1W2D3'  
console.log(parseInt(number))
```

- a. 1
- b. 1NaN2NaN3
- c. '1NaN2NaN3'
- d. '1'

13. The difference between `forEach()` dan `map()` is:

- a. `forEach()` can be used to loop through objects just like a for-loop
- b. `map()` is used if you only need a loop without creating a new array
- c. A `forEach()` loop can be stopped with `break`
- d. `forEach()` does not produce a return or a new array, whereas `map()` produces a new array

14. What is the output of the program below:



```
console.log(String(12) === '12')
```

- a. true
- b. false
- c. SyntaxError
- d. ReferenceError

15. Consider the following code, Based on the code, which time complexity best describes the code?

```
function checkDuplicate(arr: any[]): void {  
  for (let i = 0; i < arr.length; i++) {  
    for (let j = i + 1; j < arr.length; j++) {  
      if (arr[i] == arr[j]) return true;  
    }  
  }  
  return false;  
}  
  
console.log(checkDuplicate([1, 2, 3, 1]));
```

- a. $O(n)$
- b. $O(1)$
- c. $O(n^2)$
- d. $O(\log n)$